

OptiER

Sistem de distribuție a pacienților
într-o camera de gardă

Student: **Badescu Bianca-Mădălina**

Data: 4 Iunie 2026

Cuprins

1	Tematica proiectului	3
2	Descrierea structurilor de date — Arhitectura proiectului	4
2.1	Arbore Roșu-Negru (Red-Black Tree)	4
2.2	Heap Binomial (Binomial Heap)	5
2.3	Clasa Patient	5
2.4	Clasa EmergencyRoom	6
3	Explicarea algoritmilor	7
3.1	Inserare în RBT	7
3.2	Ștergere din RBT	7
3.3	Merge Heap Binomial	7
4	Analiza complexității	8
4.1	Timp	8
4.2	Spațiu	8
5	Exemple de rulare	9
5.1	Adăugare pacienți	9
5.2	Admitere pacient	9
5.3	Mass Casualty	9
5.4	Căutare pacient	9
6	Dificultăți întâmpinate	10
7	Tehnologii folosite	11

1 Tematica proiectului

Proiectul implementează un sistem de management al camerei de gardă care distribuie pacienții în mod corect și eficient între mai multe săli de tratament. Sistemul folosește două structuri de date — **Arbore Roșu-Negru** (Red-Black Tree) și **Heap Binomial** (Binomial Heap) — pentru a asigura trierea prioritară a pacienților și redistribuirea lor în caz de dezastre cu victime multiple.

Motivația alegerii temei:

- Camera de gardă din spitalele reale confruntă cu probleme de suprapopulare
- Triaajul manual poate fi incorect sau lent
- Un sistem automat bazat pe structuri de date poate asigura distribuire corectă
- Structurile alese sunt relevante pentru problema de prioritizare și fuziune

2 Descrierea structurilor de date — Arhitectura proiectului

2.1 Arbore Roșu-Negru (Red-Black Tree)

Arborele Roșu-Negru este un arbore binar de căutare auto-echilibrat care respectă 5 reguli:

1. Fiecare nod este fie **ROȘU**, fie **NEGRU**
2. Rădăcina este întotdeauna **NEAGRĂ**
3. Toate frunzele **NIL** sunt **NEGRE**
4. Dacă un nod este **ROȘU**, ambii copii sunt **NEGRI**
5. Orice cale de la un nod la frunzele **NIL** conține același număr de noduri negre

Acste reguli garantează că înălțimea arborelui este $O(\log n)$, deci toate operațiile sunt $O(\log n)$ în cel mai rău caz.

În proiect, RBT este folosit pentru:

- Indexarea tuturor pacienților după prioritatea compozită
- Căutare rapidă $O(\log n)$ a unui pacient după prioritate
- Suport pentru interogări pe interval (range queries)
- Menținerea ordinii sortate pentru afișare

Operații implementate:

Operație	Complexitate	Descriere
<code>insert(key, patient)</code>	$O(\log n)$	Inserare cu fixup (3 cazuri)
<code>delete(key)</code>	$O(\log n)$	Ștergere cu fixup (4 cazuri)
<code>search(key)</code>	$O(\log n)$	Căutare BST standard
<code>left_rotate(x)</code>	$O(1)$	Rotație la stânga
<code>right_rotate(x)</code>	$O(1)$	Rotație la dreapta
<code>insert_fixup(z)</code>	$O(\log n)$	Reparare după inserare
<code>delete_fixup(x)</code>	$O(\log n)$	Reparare după ștergere
<code>transplant(u, v)</code>	$O(1)$	Înlocuire subarbore
<code>minimum(node)</code>	$O(\log n)$	Găsire minim din subarbore
<code>to_dict()</code>	$O(n)$	Serializare pentru vizualizare

2.2 Heap Binomial (Binomial Heap)

Heap-ul Binomial este o structură de date care constă dintr-o colecție de arbori binomiali. Un arbore binomial de ordin k are exact 2^k noduri.

Proprietăți:

- Fiecare arbore binomial respectă proprietatea de min-heap (părinte $<$ copii)
- Arborii din lista de rădăcini sunt ordonați crescător după grad
- Doi arbori de același grad pot fi uniți într-un arbore de grad+1

În proiect, Heap-ul Binomial este folosit pentru:

- Câte un heap pentru fiecare sală de tratament
- Extragerea pacientului cel mai urgent din fiecare sală
- Fuziunea heap-urilor în caz de “mass casualty”
- Operația de merge este $O(\log n)$

Operații implementate:

Operație	Complexitate	Descriere
insert(key, patient)	$O(\log n)$	Fuziunea a două heap-uri Extrage minimumul și reconstruiește Găsește minimumul din lista de rădăcini Leagă doi arbori de același grad Serializare pentru vizualizare
merge(other)	$O(\log n)$	
extract_min()	$O(\log n)$	
find_min()	$O(\log n)$	
binomial_link(y, z)	$O(1)$	
to_list()	$O(n)$	

2.3 Clasa Patient

Fiecare pacient are următoarele atribute:

Atribut	Tip	Descriere
name	string	Numele pacientului
cnp	string	Cod numeric personal
severity	int (1–5)	1 = critic, 5 = minor
arrival_time	datetime	Momentul sosirii la urgență
assigned_room	int / None	Sala atribuită

Prioritatea se calculează:

$$\text{priority} = \text{severity} \times 10 - \frac{\text{minute_asteptare}}{30}$$

Aceasta asigură:

- Pacienții critici (severitate 1) au prioritate mai mare decât cei minori

- Timpul de așteptare crește gradual prioritatea
- Exemplu: pacient cu severitate 5 care așteaptă 2 ore: $\text{prioritate} = 50 - 4 = 46$
- Exemplu: pacient cu severitate 3 care tocmai a sosit: $\text{prioritate} = 30$
- Pacientul care așteaptă mult va fi tratat înainte

2.4 Clasa EmergencyRoom

Clasa principală leagă cele două structuri de date:

Atribut	Tip	Rol
<code>rbt</code>	<code>RBTree</code>	Indexare globală pacienți
<code>rooms</code>	<code>{int: BinomialHeap}</code>	Coadă de prioritate per sală
<code>patient_records</code>	<code>{str: Patient}</code>	Căutare rapidă după CNP
<code>room_sizes</code>	<code>{int: int}</code>	Contor pacienți per sală
<code>total_treated</code>	<code>int</code>	Total pacienți tratați

Fluxul de funcționare:

1. Pacientul sosește → se calculează prioritatea
2. Se găsește sala cu cele mai puține pacienți (`room_sizes` minim)
3. Se inserează în heap-ul sălii respective
4. Se inserează în RBT pentru indexare globală
5. La admitere: `extract_min` din heap + `delete` din RBT

3 Explicarea algoritmilor

3.1 Inserare în RBT

1. Se inserează nodul ca într-un BST obișnuit
2. Noul nod este colorat **ROȘU**
3. Dacă părintele este **ROȘU**, se încalcă regula 4 → se apelează `insert_fixup`

Cazurile de reparare:

Cazul 1 — Unchiul este ROȘU: Recolorăm părintele, unchiul și bunicul. Mutăm problema la nivelul bunicului.

Cazul 2 — Unchiul este NEGRU, nodul este copil drept (triunghi): Rotim părintele la stânga. Se transformă în Cazul 3.

Cazul 3 — Unchiul este NEGRU, nodul este copil stâng (linie): Recolorăm părintele NEGRU și bunicul **ROȘU**. Rotim bunicul la dreapta. Arborele devine valid.

Complexitatea totală: $O(\log n)$.

3.2 Ștergere din RBT

1. Se caută nodul de șters
2. Se aplică ștergerea BST (3 cazuri: frunză, un copil, doi copii)
3. Dacă un nod NEGRU a fost eliminat → se apelează `delete_fixup`
4. Fixup-ul are 4 cazuri (+ oglinzi) bazate pe fratele nodului problemă:

Cazul 1: Fratele w este **ROȘU** — rotim pentru a transforma

Cazul 2: Fratele w este NEGRU cu ambii copii NEGRU — mută negrimea sus

Cazul 3: Fratele w este NEGRU cu copilul apropiat **ROȘU** — rotire pregătitoare

Cazul 4: Fratele w este NEGRU cu copilul depărtat **ROȘU** — rotire finală și recolorare

3.3 Merge Heap Binomial

Algoritmul de fuziune este similar cu adunarea binară:

1. **Faza 1:** Se îmbină listele de rădăcini ordonate după grad (similar cu *merge sort* pe liste înlănțuite)
2. **Faza 2:** Se parcurge lista combinată:
 - Dacă doi arbori au același grad → se leagă (`binomial_link`)
 - Dacă trei arbori au același grad → se leagă ultimii doi, se continuă
3. Rezultatul este un heap valid cu cel mult $\lfloor \log_2 n \rfloor + 1$ arbori

4 Analiza complexității

4.1 Timp

Operație	RBT	Heap Binomial	Justificare
Inserare	$O(\log n)$	$O(\log n)^*$	* $O(1)$ amortizat
Ștergere/Extract	$O(\log n)$	$O(\log n)$	Height/sibling traversal
Căutare	$O(\log n)$	$O(n)$	BST vs list traversal
Găsire minim	$O(\log n)^{**}$	$O(\log n)$	**leftmost / root list
Merge	N/A	$O(\log n)$	Avantaj principal vs heap binar
rotație/Link	$O(1)$	$O(1)$	Operație elementară

Tabela 1: Comparație complexitate timp

Comparație cu alte structuri:

- Heap Binar: merge = $O(n)$, RBT nu suportă merge
- Heap Binomial: merge = $O(\log n)$
- AVL: mai multe rotații la inserare decât RBT

4.2 Spațiu

Ambele structuri folosesc $O(n)$ spațiu:

- RBT: 6 câmpuri per nod (key, patient, color, left, right, parent)
- Heap Binomial: 6 câmpuri per nod (key, patient, degree, parent, child, sibling)

5 Exemple de rulare

5.1 Adăugare pacienți

Input: Adăugăm 6 pacienți cu severități diferite

```
Name: Ion      | CNP: 123 | Severity: 2
Name: Maria    | CNP: 456 | Severity: 5
Name: Andrei   | CNP: 789 | Severity: 1
Name: Elena    | CNP: 321 | Severity: 3
Name: Mihai    | CNP: 654 | Severity: 4
Name: Ana      | CNP: 987 | Severity: 1
```

Rezultat:

```
Rooms: {1: 2, 2: 2, 3: 2}   # distributie egala
Waiting: 6, Treated: 0
```

Pacienții sunt distribuiți automat în sala cu cele mai puține persoane.

5.2 Admitere pacient

Input: Admit from Room 1

Rezultat:

```
Admitted: Ion (sev:2, room:1)
Rooms: {1: 1, 2: 2, 3: 2}
Waiting: 5, Treated: 1
```

Heap-ul extrage pacientul cu prioritatea minimă (cel mai urgent).

5.3 Mass Casualty

Input: Mass Casualty Mode (după admiterea unor pacienți)

Rezultat:

```
Redistributed: 5 pacienti
Rooms: {1: 2, 2: 2, 3: 1}
```

Toți pacienții sunt redistribuiți round-robin pentru echilibru.

5.4 Căutare pacient

Input: Search CNP “101”

Rezultat:

```
Name: Ion
Severity: 2 - Urgent
Room: 1
Priority: 20.0
Arrived: 14:30:25
```

6 Dificultăți întâmpinate

1. Implementarea `delete_fixup` pentru RBT

- Dificil la debug fără vizualizare

2. Algoritmul de merge pentru Heap Binomial

- Manipularea pointerilor `sibling/parent/child` este complexă
- Trebuie menținută ordinea după grad în lista de rădăcini
- Cazurile cu 3 arbori de același grad erau mai e complexe

3. Integrarea Flask cu structurile de date

- Serializarea structurilor în JSON (`to_dict`, `to_list`)
- Sincronizarea dintre RBT și Heap-uri la admitere și mass casualty

7 Tehnologii folosite

Componentă	Tehnologie
Limbaaj	Python 3.12
Framework web	Flask
Frontend	HTML5, CSS3, JavaScript
Structuri date	Implementate fără biblioteci standard

Bibliografie:

1. Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms* (CLRS), Capitolul 13 (Red-Black Trees) și Capitolul 19 (Binomial Heaps)
2. GeeksforGeeks — “Red-Black Tree Set 1, 2, 3”
<https://geeksforgeeks.org/red-black-tree-set-1-introduction-2/>
3. GeeksforGeeks — “Binomial Heap”
<https://geeksforgeeks.org/binomial-heap-2/>
4. Programiz — “Red-Black Tree” și “Binomial Heap”
<https://programiz.com/dsa/red-black-tree>
5. Flask Documentation
<https://flask.palletsprojects.com/>